



SDAIA ACADEMY · APPLIED GENERATIVE AI

Tools and agents

Day 3 · Tuesday 4 August 2026 · Musa Ibn Rashid

Let it act — but keep the leash short.

PRESENTER NOTES

Today the model stops only answering and starts doing, which is the point where this technology gets genuinely useful and genuinely risky at the same time.

The whole morning builds to one thing: an agent that uses yesterday's retriever as one of its tools.

The afternoon is project time, so this is the last full teaching morning of the week.

THE WEEK

Where we stand

SUNDAY From token to typed output — the model as a reliable function.

MONDAY Retrieval you can measure — RAG, hybrid search, and a golden set that scores it.

TUESDAY Tools and agents — the model requests, your code executes.

WEDNESDAY Production and the numbers — latency, caching, and what this costs per user.

THURSDAY Break it, then defend it — prompt injection, red teaming, and your demo.

There is no such thing as a dumb question. If you are wondering, someone else is too — ask.

PRESENTER NOTES

Two days behind you, and both of them show up in today's work.

Say the payoff clearly: Monday's retriever becomes one tool inside today's agent. That is the week joining up.

This is the last full teaching morning. From one o'clock the week turns into project time.

TODAY

How the day runs

9:15 – 10:05	Block 1 · Function calling · Be the Agent	50 min
10:05 – 10:20	Break	
10:20 – 11:00	Block 2 · Patterns, guardrails, error recovery	40 min
11:00 – 11:20	Break	
11:20 – 12:00	Block 3 · Notebook 3 — the loop, then agentic RAG	40 min
12:00 – 1:00	Lunch	60 min
1:00 – 2:25	Blocks 4 + 5 · Project time — I circulate	85 min
2:25 – 2:25	Break	
2:25 – 2:30	Stand-up · one sentence per pair	5 min

PRESENTER NOTES

Notice the shape of today: teaching stops at noon and the whole afternoon is yours.

The gate at half past two is documents loaded, chunked, embedded, and one query returning something. Running, not polished.

Anyone still stuck at two o'clock gets me sitting next to them.



DAY 3

What you will be able to do by 12:00

- 01** Explain exactly who executes a tool call, and who does not.
- 02** Write a tool schema whose description the model can actually act on.
- 03** Run the request-execute-return loop by hand, then automate it.
- 04** Put a step cap, a tool allow-list and an output validator around an agent.
- 05** Give the agent your Day 2 retriever as a tool, and read its decision trace.

PRESENTER NOTES

Number one sounds trivial and is the thing most people get wrong, including people who have shipped agents.

Number five is the payoff of the whole week — your Monday work becomes a component inside your Tuesday work.

We stop teaching at twelve today, because you need the whole afternoon on your projects.

YESTERDAY · QUESTIONS TO THE ROOM

Three questions before we start

- 01** Why did vector search fail on the reference number, and what fixed it?
- 02** What does re-ranking buy you that a bigger top-k does not?
- 03** What is a golden set actually measuring?

Ask three people who did not answer yesterday.
Then move.

PRESENTER NOTES

Keep this to five minutes. The purpose is to surface who did not follow yesterday, not to re-teach it.

If two or more people stumble on the third question, that is worth two minutes, because their project score depends on it.



PART ONE

Giving the model hands

Function calling: what it is, and who is really in control.

PRESENTER NOTES

Everything in the next hour rests on one idea about where execution happens, so I am going to repeat it more than you think is necessary.

By the end of this section you will have run the loop manually, by hand, before we let any library do it for you.

WHY BOTHER

Three things a model cannot do alone

No live data

It cannot see today's exchange rate, your ticket queue, or whether that employee is still on leave.

Its knowledge stopped at a training cut-off.

No actions

It can write a beautiful email. It cannot send one, book anything, or update a record.

Text in, text out. That is the whole interface.

Unreliable arithmetic

It predicts the next token, so "1,247 × 0.83" produces a plausible-looking number.

A calculator is right. Use the calculator.

Tools fix all three the same way: the model asks, your code does the work.

PRESENTER NOTES

The arithmetic one always gets a laugh and it deserves to be taken seriously, because the wrong number arrives formatted exactly like a right one.

Anything with a correct answer — arithmetic, dates, lookups, conversions — should be a tool, not something you hope the model gets right.

The model does not run your code.

It returns a request. Your application decides whether to honour it.

PRESENTER NOTES

I am going to say this three times today and this is the first. The model never executes anything — not your function, not a shell command, not a database query.

It emits a structured message that says "I would like calculate, with this argument". Your code reads that message and chooses what to do with it.

That is not a technicality. It is the only place where you can enforce permissions, and on Thursday it is the difference between a nuisance and a breach.

THE MECHANISM

The loop, in five steps

1 · DECLARE

You send the prompt plus a list of tools the model may request.

2 · DECIDE

It answers directly, or it returns a tool call with arguments.

3 · EXECUTE

Your code runs the function. Or refuses to.

4 · RETURN

You append the result to the conversation and send it back.

5 · REPEAT

Until it gives a final answer, or your step cap fires.

Steps 2 to 4 are one round trip. An agent is this loop with a goal and a cap on how many times it may go round.

PRESENTER NOTES

Walk left to right and put your finger on step three as you say it, because that is the step everyone skips when they describe this.

Notice that nothing here is magic — it is a while loop with a network call in it. The intelligence is in the model's choice at step two.

And notice step five is where every runaway cost story of the last two years comes from.

CODE

A tool declaration, argument by argument

```
from google.genai import types

def get_leave_balance(employee_id: str, year: int) -> dict:
    # A perfectly ordinary Python function.
    return hr_system.lookup(employee_id, year)

leave_tool = types.FunctionDeclaration(
    name="get_leave_balance",
    description=("Look up an employee's remaining annual "
                 "leave in working days for a given year. "
                 "Use when asked how much leave someone "
                 "has left."),
    parameters={
        "type": "object",
        "properties": {
            "employee_id": {"type": "string",
                             "description": "HR id, e.g. E-10482"},
            "year": {"type": "integer",
                     "description": "Four-digit year"},
        },
        "required": ["employee_id", "year"],
    },
)
```

Line by line

- **name** — what comes back in the request. Match it to your function.
- **description** — the highlighted block. This is what the model reads.
- **properties** — each argument typed, each with its own description.
- **required** — what the model must supply. Everything else is optional.

Same JSON Schema you wrote on Day 1. The idea keeps coming back.

THE FIELD THAT MATTERS

A vague description is a broken tool

Bad

"Gets leave data."

When should it be called? What does it return?

What is "leave data"? The model guesses, and calls it for sick leave, for policy questions, for anything with the word leave in it.

Good

"Look up an employee's remaining *annual* leave in working days for a given year. Use when asked how much leave someone has left. Does not cover sick leave."

Scope, units, and an explicit exclusion.

Write descriptions for a competent new colleague who cannot ask you a follow-up question.

PRESENTER NOTES

When an agent misbehaves, the first place to look is not the prompt and not the model — it is the tool descriptions.

The exclusion in the good example is the part people never write, and it is what stops a tool being called for a job it cannot do.

Tell them: if your agent keeps calling the wrong tool, you have a documentation problem, not an intelligence problem.

Doing it by hand, once

```
tools = types.Tool(function_declarations=[leave_tool])
cfg = types.GenerateContentConfig(tools=[tools])

# 1 · Ask. The model does NOT answer the question.
r = client.models.generate_content(
    model=MODEL, contents=history, config=cfg)

# 2 · Inspect what came back: a REQUEST, not an answer.
call = r.candidates[0].content.parts[0].function_call
print(call.name)    # get_leave_balance
print(call.args)   # {'employee_id': 'E-10482', ...}

# 3 · YOUR code executes it. Check permissions here.
result = get_leave_balance(**call.args)

# 4 · Hand the result back and ask again.
history.append(r.candidates[0].content)
history.append(types.Content(role="user", parts=[
    types.Part.from_function_response(
        name=call.name, response={"result": result})]))

final = client.models.generate_content(
    model=MODEL, contents=history, config=cfg)
```

Watch step 3

Nothing ran until *you* called the function. If you delete that line, nothing happens at all.

This is also where you check whether this user is allowed to look up that employee.

In the notebook these four steps are four separate cells, so the round trip is impossible to miss.

The agent loop

```

def run_agent(goal, tools, max_steps=5, verbose=True):
    history = [user_message(goal)]

    for step in range(max_steps):
        r = client.models.generate_content(
            model=MODEL, contents=history, config=cfg)
        part = r.candidates[0].content.parts[0]

        # No tool requested? It's finished.
        if not getattr(part, "function_call", None):
            return r.text

        call = part.function_call
        if verbose:
            print(f"step {step}: {call.name}({dict(call.args)}")

        # Allow-list: never call by name blindly.
        fn = TOOLS.get(call.name)
        result = fn(**call.args) if fn else {"error": "unknown tool"}

        history.append(r.candidates[0].content)
        history.append(tool_result(call.name, result))

    return "Stopped: step limit reached."

```

Four guardrails, already here

- `max_steps` — the loop is bounded.
- `TOOLS.get` — an allow-list, not `eval`.
- Unknown tool returns an *error the model can read*.
- `verbose` — you can see every decision it made.

Given complete in the notebook. Read it, don't retype it.

WHAT GOES WRONG

Take the cap off and watch

```
step 0: search_flights(dest="JED") step 1:  
search_flights(dest="JED") step 2:  
search_flights(dest="Jeddah") step 3:  
search_flights(dest="JED") step 4:  
search_flights(dest="JED ") step 5: ...
```

A goal it cannot satisfy, a tool that keeps returning nothing, and no reason to stop.

What that costs

- Every step is a paid call, and the history grows each time — so **each step costs more than the last**.
- Left overnight, this is the invoice that ends up in a post-mortem.
- On a free tier you hit a rate limit and stop. That is luck, not design.

Every agent gets a cap. No exceptions.

PRESENTER NOTES

We do this deliberately in the notebook — you remove the cap, run it, watch it spiral, and interrupt the cell yourself.

The detail people miss is that the conversation grows with every step, so step twenty is far more expensive than step one. The cost curve is not flat.

Ask the room what a sensible cap is. The answer is usually between three and ten, and it should be based on how many steps the task actually needs.

Six shapes, and when to use them

ReAct

Think, act, observe, repeat.

Use when: the next step depends on what you just found.

Plan-and-Execute

Plan all steps first, then run them.

Use when: the task is predictable and you want the plan reviewable.

Reflection

Produce, critique its own output, revise.

Use when: quality matters more than latency or cost.

Routing

A cheap classifier picks the specialist.

Use when: you have several narrow jobs, not one broad one.

Hierarchical

A manager agent delegates to workers.

Use when: genuinely separable sub-tasks. Rarely. It is expensive.

Human-in-the-loop

It pauses for approval before acting.

Use when: the action is irreversible. Money, messages, deletions.

Start with ReAct and a step cap. Add a pattern only when you can name the failure it fixes.

PRESENTER NOTES

One line each and keep moving; this slide is a reference they will come back to, not something to teach in depth.

Linger only on the last one. Anything irreversible — sending, paying, deleting — gets a human in the loop, and no amount of model quality changes that.

Hierarchical is fashionable and usually wrong. Say so.

CONTROL

Guardrails, concretely

Before the call

- **Step cap** — a bounded loop, always.
- **Tool allow-list** — dictionary lookup, never dynamic dispatch on a model-supplied name.
- **Least privilege** — read-only tools where reading is enough.
- **Argument validation** — the model can produce any string it likes.

After the call

- **Output validation** — check the shape before you act on it.
- **Stop conditions** — budget, wall clock, repeated identical calls.
- **Human approval** — for anything you cannot undo.
- **A trace** — every decision logged, so you can explain what happened.

The agent is the easy part. This slide is the work.

PRESENTER NOTES

Building an agent takes an afternoon. Making one you would leave running takes weeks, and it is all on this slide.

The repeated-identical-call stop condition is worth calling out — it catches the spiral we just watched, cheaply.

And the trace is what you will want at nine in the morning when somebody asks why the system did something strange.

CONTROL

What a tool returns when it fails

Bad

```
raise ValueError("not found")
```

The exception kills your loop, or worse, you swallow it and the model sees nothing and invents an answer.

Good

```
{"error": "No employee with id E-99999. Ids look like E-10482. Ask the user to confirm."}
```

The model can read this, correct itself, and recover — which is the whole reason the loop exists.

Error messages are prompts. Write them for the model, not for your log file.

PRESENTER NOTES

This is a small idea that changes how well an agent behaves, and almost nobody does it.

Your tool's failure message is the only thing the model has to work with when something goes wrong, so tell it what was wrong and what a valid input looks like.

In the Be the Agent activity you will hit exactly this — a tool returns an error and you have to recover from it. That recovery is worth points.



PART TWO · THE CEILING

Agentic RAG

Yesterday's retriever becomes today's tool.

PRESENTER NOTES

Say this one clearly, because it is the moment the week joins up.

Everything you built yesterday — chunking, hybrid search, re-ranking — is about to become a single function that an agent can choose to call, or not.

THE CEILING

Retrieval as one tool among several

Plain RAG

Always retrieves. Always once. Even when the question needs no documents at all, or needs three different searches.

Agentic RAG

The model decides *whether* to search, *what* to search for, and whether the first result was good enough to answer from.

What that unlocks

- "What is our leave policy, and how many days would I have left after two weeks off?" — retrieval *and* arithmetic.
- A bad first search can be retried with better terms.
- Questions that need nothing from the corpus skip retrieval entirely.

Cost: unpredictable calls per question. It is a trade, not an upgrade.

PRESENTER NOTES

Be honest about the trade here. Agentic retrieval is more capable and less predictable, and predictability is worth a lot in a government context.

My advice for your projects: build plain retrieval first, prove it works with your golden set, then add the agent on top if you actually need multi-step behaviour.

Your retriever, as a tool

```
# Yesterday's function. Unchanged.
from day2 import hybrid_search

def search_documents(query: str, k: int = 4) -> list:
    # Return text AND the citation, so the answer can cite.
    return [{"text": r["text"][:600],
            "source": r["source"],
            "page": r["page"]}
            for r in hybrid_search(query, k=k)]

search_decl = types.FunctionDeclaration(
    name="search_documents",
    description=("Search the organisation's policy "
                "documents and return the most relevant "
                "passages with their source and page. "
                "Use for any question about internal "
                "policy, procedure or entitlement."),
    parameters={"type": "object",
                "properties": {"query": {"type": "string",
                                         "description": "Search terms, not the raw question"}},
                "required": ["query"]})

TOOLS = {"search_documents": search_documents,
         "calculate": calculate}
```

Three things to notice

- Yesterday's code is **imported, not rewritten**.
- The truncation to 600 characters keeps the loop affordable.
- "Search terms, not the raw question" is query rewriting, delegated to the model.

THE CEILING

Reading the trace

```
goal: "What is the annual leave for grade 11, and what would I have left after taking 12 days?" step 0:  
search_documents(query="annual leave entitlement grade 11") → "Grade 11 – 30 days" [leave_policy.pdf, p.4] step 1:  
calculate(expression="30 - 12") → 18 final: "Grade 11 is entitled to 30 working days of annual leave (leave_policy.pdf, page  
4). After 12 days you would have 18 left."
```

Two tools, chosen in the right order, by the model
— and a citation in the answer.

PRESENTER NOTES

Read this trace out loud, line by line. It is the clearest picture of the week that exists.

Step zero is Monday's work. Step one is a tool because we do not trust the model with arithmetic. The final line carries a citation because we kept the metadata on Monday morning.

Every decision on this screen was the model's, and every execution was yours.

JUDGEMENT

When not to build an agent

Do not, when...

- The task is one retrieval and one answer.
- The path is always the same — that is a script, and scripts are testable.
- The action is irreversible and nobody is checking.
- You cannot afford unpredictable per-question cost.

Do, when...

- The next step genuinely depends on what the last step returned.
- Several tools are needed and you cannot know which in advance.
- The system must recover from partial failure on its own.

"Could I write this as a fixed sequence?" If yes, write the fixed sequence.

PRESENTER NOTES

This slide will save somebody in this room a quarter of their project time.

Agents are the most interesting thing we cover and the least often the right answer. Both of those are true at once.

For your project, an agent is not required for full marks. A justified architecture is.

LAB · NOTEBOOK 3

Build the loop yourself

day3_agents.ipynb

- Two real tools, then the schemas.
- The round trip by hand, in four separate cells.
- `run_agent` with a step cap and a printed trace.
- Remove the cap and watch it spiral — deliberately.
- Your Day 2 retriever registered as `search_documents`.

Then: project time, 1:00–2:30

The gate by 2:30: documents loaded, chunked, embedded, and one query returning something. Not polished — running.

Anyone still stuck at 2:00 gets me sitting next to them.

[Open the notebooks →](#)

PRESENTER NOTES

The spiral cell is deliberate and I want you to actually run it, then interrupt it yourself. Watching it happen once is worth more than the warning.

This afternoon I am not presenting anything. I will be circulating, and I will check on every pair at least twice.

We finish with a five minute stand-up: one sentence per pair on where you got to.



DAY 3

What today was

- 1 • The model requests. Your code executes. That boundary is where all your control lives.
- 2 • The tool description is the interface. Vague description, broken tool.
- 3 • An agent is a bounded loop. The cap, the allow-list and the trace are not optional extras.
- 4 • Yesterday's retriever is now a tool — and that is the whole week joining up.

PRESENTER NOTES

Four things, and the first one is the one I have said three times today on purpose.

Tomorrow we stop adding capability and start measuring what you have: latency, tokens, cost, and a cache hit rate from your own project.

Bring your project running, even if it is ugly. Tomorrow only works if there is something to instrument.